

Lecture 13

PINNs for Geometry II & Neural Operators

AI for Geometry — MM845

Edward Hirst

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas

`ehirst@unicamp.br`

Setembro 2026

Overview

1. Time-Dependent & Nonlinear PDEs
2. Curing the Pathologies
3. Neural Operators
4. Summary

Recap (Lecture 12): PINNs – PDE and boundary losses, autodiff, inverse problems.

Guiding question: *How can we incorporate time into our PINN approach?*

How do we learn solution maps that can be reused across geometric problems?

Adding Time: Space-Time PINNs

Learn the full space-time

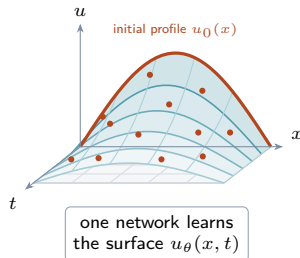
Consider PDEs of the form: $\partial_t u = \Delta u$ on $\Omega \times [0, T]$; time t is now just another input.

→ Still traditional with boundary conditions, with a special $t = 0$ one: $u(\cdot, 0) = u_0$.

→ Use a network $u_\theta(x, t)$ and train on

$$L = \lambda_r \|\partial_t u_\theta - \Delta u_\theta\|_{\Omega \times [0, T]}^2 + \lambda_0 \|u_\theta(\cdot, 0) - u_0\|_{\Omega}^2 + \lambda_b L_{bc}.$$

- **Whole trajectory:** A single function such that can query any (x, t) . A space-time residual avoids an explicit time-stepping grid.
- **Propagation failure:** fitting all times at once does not make the optimiser learn the initial state before later dynamics.
- **Validate:** compare to short-time classical reference solves, and physical checks such as energy decay for heat flow.
- **Alternatives:** train in successive time windows, or use a discrete-time PINN with a classical stepping scheme.



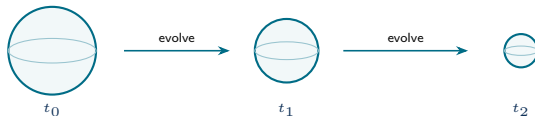
Geometric Flows: the Prize Targets

Flows of immersion architectures

Consider an immersion $X : \Sigma \times [0, T) \rightarrow \mathbb{R}^3$, which evolves via mean curvature flow by

$$\partial_t X = -H\nu = \Delta_{g(t)} X,$$

...for $g(t)$ induced by X , and mean curvature $H \implies$ the operator evolves too.



shrinking spheres before extinction: $R(t)^2 = R_0^2 - 4t$

- **Parametric:** Learn full $X_\theta(p, t)$ via residual $\partial_t X_\theta - \Delta_{g_\theta} X_\theta$. Control the parametrisation to avoid distorted coordinates.
- **Level set:** Represent $\Sigma_t = \{\phi_\theta = 0\}$ and fit $\partial_t \phi = |\nabla \phi| \operatorname{div}(\nabla \phi / |\nabla \phi|)$. This representation can accommodate topology changes, with care about the solution concept.
- **Equivalent formulations:** curve shortening, surface Ricci flow, harmonic map heat flow.

Resolve pre-singular behaviour

Singularities may need rescaling, adaptive sampling, or weak formulations.

Fully Nonlinear Problems

Nonlinear PDEs

Consider the non-linear Monge-Ampere PDE: $\det(\nabla^2 u) = f$

...with u convex and appropriate boundary data.

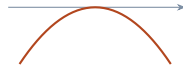
→ The residual is nonlinear in second derivatives; its linearisation changes as the network trains.

- For MA, a small determinant residual alone does not enforce convexity.
- For the CY case (L11), the Kähler-potential ansatz still requires positivity of the resulting metric.
- Build constraints into the architecture when possible (e.g. input-convex networks), or penalise and test violations.
- Classical residuals assume sufficient smoothness of the solution. If the correct solution is non-smooth the losses should use weak or viscosity formulations that target the intended solution concept.

$$u_+(x) = \frac{1}{2} \|x\|^2: \text{convex}$$



$$u_-(x) = -\frac{1}{2} \|x\|^2: \text{concave}$$



In two dimensions, both have
 $\det \nabla^2 u_{\pm} = 1$.
Curves show one-dimensional sections.

Often many solution branches, but just one physical

E.g. Optimal transport needs the convex solution; geometric metric problems need positivity.

→ Optimiser is unaware of the physicality, so build into architecture, and test for after.

Cures I: Balancing and Causality

Balance the contributions to training

- Adapt loss weights using gradient norms $\|\nabla_{\theta} L_k\|$ so each term influences training.
- Neural tangent kernel (NTK) methods estimate relative training rates.

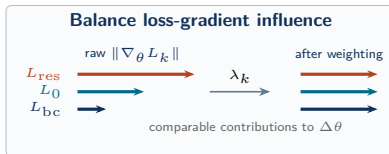
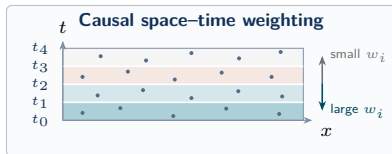
Solve in stages

For time slices with mean squared residuals r_i , one causal weighting is

$$w_i = \exp\left(-\epsilon \sum_{j < i} r_j\right), \quad L_{\text{res}} = \sum_i w_i r_i, \quad \epsilon > 0.$$

...such that large early residuals suppress the weight of later slices.

- Initial data must also be fitted and monitored.
- Practically can solve in time windows with each initialised from the preceding solution.
- Expand beyond time slices to slices in another parameter,
...using a curriculum over say coefficients to increase nonlinearity or stiffness gradually.



Cures II: Spectral Bias and Decomposition

Fourier features: expose the useful spatial scales

A plain network tends to learn smooth, low-frequency structure first.

→ Embed $x \mapsto \Phi_B(x) = (\sin(2\pi Bx), \cos(2\pi Bx))$,

...where the rows of B supply frequencies chosen to resolve the expected solution.

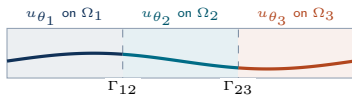
XPINN: explicit interfaces

Split $\Omega = \bigcup_i \Omega_i$

...train one PINN u_{θ_i} per subdomain.

→ Each fits the PDE locally; on Γ_{ij} match $u_{\theta_i} \approx u_{\theta_j}$ and $\partial_n u_{\theta_i} \approx \partial_n u_{\theta_j}$.

→ Smaller local problems can follow different scales and train in parallel, but the joins must be monitored explicitly.



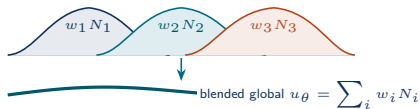
explicitly match values and required fluxes at interfaces

FBPINN: overlapping windows

Give local networks overlapping windows w_i forming a partition of unity:

$$u_{\theta}(x) = \sum_i w_i(x) N_i(x), \quad \sum_i w_i(x) = 1.$$

→ The smooth windows where each N_i learns are blended into one global prediction.



Neural Operators

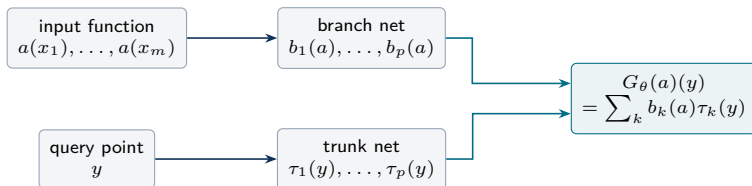
Learn a map between function spaces

A typical PINN fits one PDE instance. A **neural operator** shares work across a family:

$$G : a \mapsto u_a, \quad \text{or} \quad G : u_0 \mapsto u(\cdot, T).$$

- Train on many input/solution pairs (often produced by classical solvers).
- The n -th data pair is: (input, output) = $([a^n(x_1), a^n(x_2), \dots, a^n(x_m)], y), u^n(y)$.
- Evaluate the learned map on new inputs, y' .

DeepONet:



- The branch encodes the input function at points $\{x_i\}$.
 - The trunk supplies learned basis functions at the output location.
 - $\implies$ The expansion has input-dependent coefficients and a learned basis.
 - Useful for parameter sweeps, repeated inverse solves; test on held-out *input functions*.
- Operator approximation theorems motivate it, under suitable assumptions.

Fourier Neural Operators

An FNO layer combines local and spectral operations

FNO architecture: $a(x_i) \xrightarrow{P} v_0(x_i) \xrightarrow{\text{FNO layers}} v_L(x_i) \xrightarrow{Q} u(x_i),$

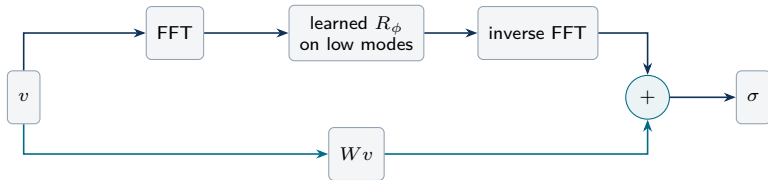
... P, Q are simple MLPs that introduce channel dimensions per grid point.

FNO layer: Feature field v (discretised over grid $[v(x_i)]$), Fourier transform $\mathcal{F}$, nonlinearity σ ,

$$v \mapsto \sigma\left(Wv + \mathcal{F}^{-1}\left(R_\phi \mathcal{F}v\right)\right).$$

...for learnt matrix W mixing channels pointwise;

...and learnt R_ϕ mixing channels within each retained Fourier mode.

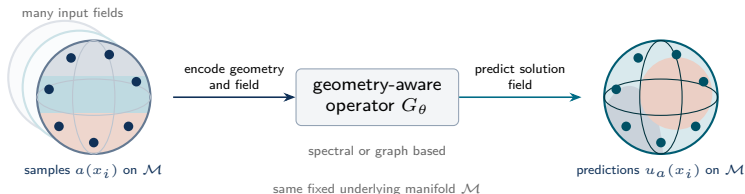


- **Global interaction:** Spectral features are naturally global (FFT costs $O(N \log N)$).
- **Resolution transfer:** The same (W, R_ϕ) weights can be used on another grid.
- **Amortised cost:** Training needs data and computation; subsequent predictions are cheap. Physics-informed operators also penalise PDE residuals.

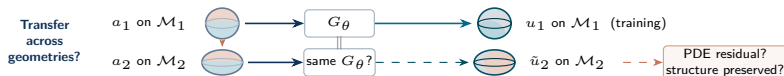
Operators on Manifolds & the Research Frontier

Replace the grid representation with geometric structure

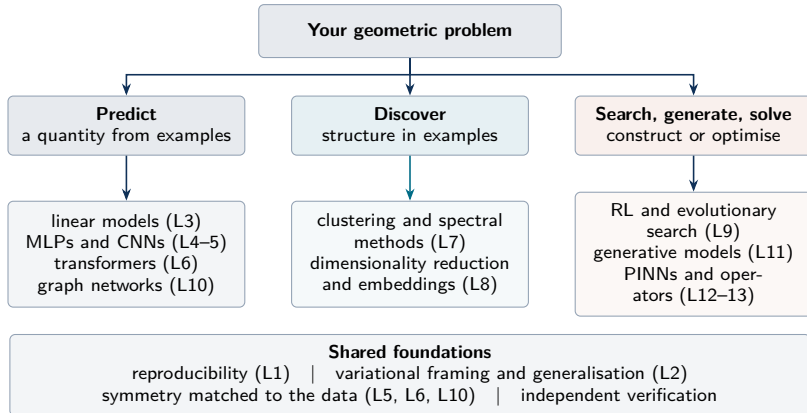
- On a fixed manifold $(\mathcal{M}, g)$, Laplace–Beltrami eigenfunctions can replace Fourier modes.
- Graph-based operators work with irregular discretisations.



- **Flow surrogates:** emulate short steps of a geometric evolution, then validate interesting trajectories with trusted numerical solvers.
- **Moduli exploration:** learn maps from geometric parameters to numerical solutions, such as CY metrics, to speed up repeated queries.
- **Open questions:** approximation rates, transfer across geometries, basis alignment, and preservation of conservation laws or other structure.



The Toolkit, Assembled



The course perspective

What is your task? What does your data look like?

- Choose your *method* to match your data.
- Choose your *architecture* to match your symmetries / properties.
- Choose your *loss* to match your goal.

Summary — and Farewell

Key points

- PINNs can be adapted for space-time evolution and geometric flows;
...balance losses, apply causal weighting, resolve relevant frequencies,
...and use domain decomposition where helpful; verify improvements in solution error.
- Neural operators learn reusable solution maps:
DeepONet uses branch/trunk networks, FNO spectral multipliers; both extend to manifolds.

Mini-project guidance

Workflow: Focus on one clear question, build a meaningful dataset, choose an appropriate method/architecture, compare against a simple baseline, explain and independently verify results.

Deliver:

- A 10-min presentation (01/10).
- A 5-page report (04/10).
- A reproducible Git repository (environment, seeds, README, code).

...a modest, well-understood result is enough; as is a negative result with clear diagnosis.

Next

Tutorial 13: Time-dependent geometric PDEs.

Future: Projects & presentations $\mapsto$ Boa sorte! 😊